

Library Book Management System

Version 2.0

VITyarthi — Build Your Own Project

Subject: Programming in Java

Project Report — Academic Year 2024–25

1. Introduction

This report documents the design, development, and evaluation of the Library Book Management System — a console-based Java application developed as part of the Programming in Java flipped course evaluation at VITyarthi.

The application allows library staff to manage books, members, borrow/return transactions, book reservations, overdue fines, and generate analytical reports — all through a secure, menu-driven interface. Data is persisted to flat files so the library's state survives application restarts.

Core Java Concepts Applied

- Object-Oriented Programming: encapsulation, single-responsibility classes, clean separation of concerns
- Java Collections Framework: `HashMap<String, Book>`, `ArrayList<Transaction>`
- Stream API: `filter()`, `map()`, `collect()`, `reduce()` for search and reporting
- `java.time.LocalDate`: due date setting and overdue day calculation via `ChronoUnit`
- `java.io`: `BufferedReader` / `BufferedWriter` for file persistence
- Enum types: `Transaction.Type` (BORROW / RETURN)
- `AtomicInteger`: thread-safe sequential ID generation
- Regex validation: `String.matches()` for email and phone
- Password hashing: `String.hashCode()` for admin credential security

2. Problem Statement

Libraries relying on manual paper-based records face recurring operational problems:

- No real-time visibility into which books are available or who currently holds them
- Overdue books go unnoticed for weeks, resulting in loss of resources and untracked financial penalties
- No systematic mechanism for members to queue for a book that is already issued
- Zero access control — any person can view or modify records without authentication
- Manual record-keeping makes generating management reports slow and error-prone
- Data is lost if records are damaged or misplaced

This system addresses every one of these issues with a secure, automated Java application backed by file-level persistence and a comprehensive reporting module.

3. Objectives

- Build a fully functional library management system demonstrating Java OOP principles
- Implement four functional modules: Book Management, Member Management, Borrow/Return, and Admin Authentication
- Add a reservation system so members can queue for issued books
- Implement an automatic fine calculator (Rs. 2/day overdue)
- Provide file persistence so data survives application restarts
- Write clean, modular, well-commented code with at least 5–10 meaningful classes

- Include a comprehensive test suite covering all modules

4. Functional Requirements

Module	Feature	Details
Admin Auth	Login	Username + hashed password; 3-attempt lockout
Admin Auth	Logout	Clears session; returns to login screen
Book Mgmt	Add Book	Title, author, genre, year; auto-generates BK-NNNN ID
Book Mgmt	Remove Book	Only if currently available (not issued)
Book Mgmt	Update Book	Update title, author, genre by book ID
Book Mgmt	Search	By title / author / genre, case-insensitive partial match
Book Mgmt	List	All books or available-only filter
Member Mgmt	Register	Name, email (validated), phone (10-digit validated), duplicate email check
Member Mgmt	Update	Update name, email, phone
Member Mgmt	Remove	Only if no active borrowed books
Borrow	Issue Book	Validates: member exists, book available, borrow limit < 3, reservation priority
Borrow	Due Date	Set to issue date + 14 days automatically
Borrow	Return Book	Marks book available, detects and reports overdue fine
Reservation	Reserve	Reserve an issued book; stored on the Book entity
Reservation	Cancel	Cancel an active reservation
Reservation	Priority Issue	Reserving member gets priority when book is returned
Fine Calc	Per-Member	Shows fine per overdue book and total payable
Fine Calc	Rate	Rs. 2 per day overdue
Reports	Full History	All borrow/return transactions ever recorded
Reports	Member History	Filtered by member ID
Reports	Overdue Report	All currently overdue borrow

		transactions
Reports	Summary	Total books, available, issued, members, borrows, returns, overdue count

5. Non-Functional Requirements

#	Requirement	Implementation
1	Performance	O(1) lookup via HashMap for books and members; Stream API for filtering
2	Security	Admin login required; passwords stored as hashCode(), 3-attempt lockout
3	Usability	Clear numbered console menus, ✓/✗ prefixes on success/failure messages
4	Reliability	Input validated before storage; corrupt file records skipped gracefully
5	Maintainability	Single-responsibility classes; 6 packages; all public methods documented
6	Scalability	Data layer (FileStorage) can be swapped to JDBC without changing service logic
7	Logging	Logger writes timestamped entries to both console and data/app.log
8	Error Handling	Null checks, borrow limit guards, issuance constraint — all with user messages
9	Persistence	All data auto-saved after every mutating operation; loaded on startup
10	Testability	21 test cases in LibrarySystemTest.java covering all modules

6. System Architecture

The project uses a layered MVC-inspired architecture with five distinct layers, each with a clear responsibility:

Layer	Package	Classes	Responsibility
-------	---------	---------	----------------

Presentation	library.ui	ConsoleMenu	All user interaction; collects input, displays output
Business Logic	library.service	BookService, MemberService, BorrowService, AdminService	Core application rules and workflows
Data Model	library.model	Book, Member, Transaction, Admin	Entities with state and serialisation
Persistence	library.storage	FileStorage	Read/write flat .txt files; decoupled from services
Utilities	library.util	Validator, IdGenerator, Logger	Shared cross-cutting concerns

Data flow: User Input → ConsoleMenu → Service Layer → Model Objects → FileStorage → disk.
On startup, FileStorage loads data into service maps before the menu is shown.

7. Design Diagrams

7.1 Use Case Diagram

Shows the two actors (Admin/Staff and Member) and all use cases with include and extend relationships.

[Diagram: Use Case Diagram] — See file: docs/diagrams/use_case.svg

The Admin/Staff actor has access to all six use cases. The Member actor interacts only with the reservation use case indirectly through staff. The Login use case is included by all others — no feature is accessible without authentication.

7.2 Class Diagram

Shows all 13 classes across 5 packages with fields, methods, and relationships (association, dependency, uses).

[Diagram: Class Diagram] — See file: *docs/diagrams/class_diagram.svg*

BorrowService holds references to both BookService and MemberService. All three service classes hold a dependency on FileStorage (via persist()). Model classes are pure POJOs with serialisation methods. Utility classes are all-static.

7.3 Sequence Diagram — Issue Book

Shows the complete message flow when a staff member issues a book, including all validation checks and the persistence step.

[Diagram: Sequence Diagram — Issue Book] — See file: docs/diagrams/sequence_issue.svg

The sequence passes through 8 steps: input → validate member → validate book → check reservation → check borrow limit → mark issued → log transaction → persist. Each failure path exits early with an informative message.

7.4 Workflow / Process Flow

Shows the full user journey from startup through login to all six menu branches and back, including the auto-persist step.

[Diagram: Workflow Diagram] — See file: <docs/diagrams/workflow.svg>

7.5 ER Diagram

Shows the three main data entities (Book, Member, Transaction) with attributes, primary keys, foreign keys, and constraints.

[Diagram: ER Diagram] — See file: docs/diagrams/er_diagram.svg

The M:N relationship between Member and Book is resolved through the Transaction entity, which records every borrow and return event. Key constraints: email UNIQUE, phone 10-digit, isAvailable default true, dueDate NULL for RETURN transactions.

8. Design Decisions & Rationale

Decision	Alternative Considered	Rationale
HashMap for book/member storage	ArrayList with linear search	O(1) lookup vs O(n); critical for performance at scale
Flat file persistence (txt)	SQLite via JDBC	No external dependencies; easier to inspect and demo; JDBC planned for v3
Pipe-delimited serialisation	JSON / XML	No external library needed; Java standard library handles it
Static utility classes	Singleton pattern	No state needed; static is simpler and cleaner for pure utility logic
AtomicInteger for IDs	UUID	Human-readable, sequential IDs are better for a console demo and manual testing
String.hashCode() for passwords	Plaintext storage	Never store plaintext; hashCode is simple and sufficient for this scope
3-book borrow limit	No limit	Prevents monopolisation; mirrors real library policy; easily configurable constant
14-day loan period	7 or 30 days	Standard for academic libraries; defined as a single constant for easy adjustment
BorrowService coordinates services	Direct coupling	Single point of coordination avoids circular dependencies between Book and Member services

9. Implementation Details

File & Package Structure

File	Package	Lines	Role
Main.java	library	24	Entry point; initialises storage, launches menu
Book.java	library.model	84	Book entity with reservation field and serialisation
Member.java	library.model	75	Member entity with borrow list and serialisation
Transaction.java	library.model	94	Borrow/return record with fine calculation
Admin.java	library.model	38	Admin account with hashed password

BookService.java	library.service	170	Module 1 — book CRUD, search, reservation
MemberService.java	library.service	111	Module 2 — member management with validation
BorrowService.java	library.service	202	Module 3 — issue, return, fine, reporting
AdminService.java	library.service	63	Module 4 — login, logout, session
FileStorage.java	library.storage	125	Persistence layer — read/write all .txt files
ConsoleMenu.java	library.ui	282	Full interactive menu for all 6 modules
Validator.java	library.util	45	Email, phone, year, ID validation
IdGenerator.java	library.util	23	Thread-safe sequential ID counters
Logger.java	library.util	50	Timestamped console + file logging
LibrarySystemTest.java	test/	~220	21 test cases across all modules

Total: 14 source files, ~1600 lines of clean, documented Java code across 6 packages.

Key Implementation Highlights

- Stream API used in 12+ locations for filtering, searching, aggregating, and reducing data
- Every mutating operation in service classes calls `persist()` immediately — no data is ever lost if the app crashes
- `FileStorage.initialise()` creates missing files/directories on first run — zero manual setup needed
- The reservation system is implemented as a nullable field on `Book` — clean, simple, effective
- `Logger` writes to both `System.out` and `data/app.log` simultaneously — useful for post-session audit
- Switch expressions (Java 14+) used in `ConsoleMenu` for concise menu routing without fall-through risk

10. Testing Approach

A manual test suite (`LibrarySystemTest.java`) covers 21 test cases across all modules. Each test creates a fresh service instance, performs an action, and asserts the expected outcome with a PASS/FAIL label.

Test ID	Test Case	Expected Result
T01	Add new book	Book added, catalogue size +1

T02	Remove available book	Book deleted from catalogue
T03	Remove issued book	Rejected with error message
T04	Search by title	Returns matching books
T05	Search by author	Returns correct results
T06	Update book details	New title/author reflected
T07	Register valid member	Member created with generated ID
T08	Invalid email rejected	Registration returns null
T09	Short phone rejected	Registration returns null
T10	Duplicate email rejected	Second registration returns null
T11	Remove member with borrows	Rejected with active borrow message
T12	Issue available book	Book marked issued, true returned
T13	Book unavailable after issue	isAvailable() returns false
T14	Issue already-issued book	Rejected, false returned
T15	Borrow limit enforcement	4th borrow attempt rejected
T16	Return book	Book marked available, true returned
T17	Book available after return	isAvailable() returns true
T18	Reserve issued book	isReserved() returns true
T19	Correct member ID stored	getReservedByMemberId() matches
T20	Admin correct login	isLoggedIn() returns true
T21	Admin wrong password	isLoggedIn() returns false

11. Screenshots / Results

Sample console output for key workflows:

Login Screen

```

┌────────────────────────────────────────────────────────────────────────────────┐
│                                     ADMIN LOGIN                                │
└────────────────────────────────────────────────────────────────────────────────┘
Default: admin / admin123
Username : admin Password : admin123 [INFO ] 2025-01-15 10:23:01 – Admin logged in:
admin ✓ Welcome, admin! Login successful.

```

Issue Book

```

Choice: 3 — Borrow / Return — 1. Issue Book  2. Return Book  0. Back Choice: 1
Member ID : MB-2000 Book ID   : BK-1002 ✓ Issued: "To Kill a Mockingbird" → Arjun
Sharma   Due date: 2025-01-29

```

Fine Summary

```

— Fine Summary for Arjun Sharma —   Book BK-1002   | Due: 14-01-2025 | Fine: Rs.
2.00   Total Fine Payable: Rs. 2.00

```

Library Summary

LIBRARY SUMMARY			
Total Books		: 9	Available Books : 7
Currently Issued : 2	Reserved Books : 1	Registered Members: 5	
Total Returns : 6		Total Borrows : 4	Overdue Items : 1

12. Challenges Faced

- Coordinating BorrowService across BookService and MemberService without circular dependencies — solved by injecting both services via constructor
- File persistence required designing a custom pipe-delimited serialisation format that handles null values and list fields cleanly
- The reservation system interacts with the issue flow — a reserved book must only be issuable by the reserving member, requiring a careful multi-condition check
- Overdue detection using `LocalDate.isAfter()` needed careful edge-case handling (same-day return is not overdue)
- Making the Logger write to a file while `FileStorage.initialise()` also creates the `data/` directory — required ordering startup steps carefully in `Main.java`

13. Learnings & Key Takeaways

- A layered architecture with clear package boundaries makes code dramatically easier to extend, test, and debug
- The Stream API eliminates the need for verbose for-loops in filtering and aggregating collections — writing `.filter().collect()` is both cleaner and less error-prone
- Input validation at the service layer (not the UI layer) is the right place — it keeps the model clean regardless of how the UI changes
- File persistence forces careful thought about serialisation format — pipe-delimited format works well but requires escaping if field values might contain the delimiter
- `java.time` is modern and elegant — `LocalDate.plusDays()` and `ChronoUnit.DAYS.between()` are far cleaner than Calendar arithmetic
- Writing tests first (or at least alongside) reveals design flaws early — the borrow-limit and reservation tests caught edge cases that were not initially considered

14. Future Enhancements

Enhancement	Benefit
JDBC + SQLite persistence	Proper relational storage; faster queries; ACID compliance
Fine payment tracking	Record whether fines have been paid; generate receipts
Book categories and tags	More granular search and classification
Multiple copy tracking	Support for libraries that own more than one copy per title
JavaFX GUI	Graphical interface replacing the console menu
REST API with Spring Boot	Web access for remote staff and mobile apps

Email notifications	Notify members when reserved books become available
Role-based access control	Separate admin vs read-only staff permissions

15. References

- Bloch, J. (2018). Effective Java, 3rd Edition. Addison-Wesley Professional.
- Horstmann, C. S. (2021). Core Java Volume I — Fundamentals, 12th Edition. Prentice Hall.
- Oracle Corporation. Java SE 17 Documentation. <https://docs.oracle.com/en/java/javase/17/>
- Oracle Corporation. Java Collections Framework. <https://docs.oracle.com/javase/tutorial/collections/>
- Oracle Corporation. java.time API. <https://docs.oracle.com/javase/8/docs/api/java/time/package-summary.html>
- VITyarthi. Build Your Own Project — General Project Instructions & Submission Guidelines. 2024.